

QA Strategy: WeDance 2026 (2026.wedance.vip)

Version 1.0 | Last Updated: 2026-10-09 | Owner: Vitali (proposed — confirm in RAZ-268)

1. Executive Summary

WeDance 2026 is a Nuxt 4 + tRPC + Drizzle/Neon dance-community platform with about 3,900 migrated dancer accounts, about 1,300 upcoming events mirrored daily from wedance.vip, and real money flowing through TicketTailor tickets and Stripe referral discounts. It ships continuously: 97 PRs were merged to main in the last 30 days, almost all written by AI agents and merged by an automated gate. That gate has nothing to check except the Vercel build, because the repo has **no CI test run**: the 242 unit tests run only when someone runs them locally, and **3 of them are failing on main today** without anyone noticing. The E2E layer is empty — the Playwright-BDD config points at feature files outside the repo, a path broken since the repo was flattened. In the last 60 days, 15 real defects reached production; the largest class (5 of 15) was sign-in and access control ("I can click Going without being signed in", "Forgot password doesn't reset the password"). This strategy adds the missing layers in risk order: first a required CI check so the merge gate can see red, then journey-level E2E tests for the CRITICAL flows (sign-in, access control, tickets, deploy health), then DB-backed integration tests. Headline targets for 2027-01-31: zero failing tests on main, all 8 user journeys with at least one passing E2E test, PR feedback under 10 minutes, and escaped defects below 4 per month (baseline ~7.5).

2. Scope & Objectives

In scope

- The razbakov/wedance-2026 repo: Nuxt pages and components (app/), the 24 tRPC routers (server/trpc/routers/), REST endpoints and webhooks (server/api/ — Stripe, TicketTailor, festivals, cities), shared utils and validation (shared/), DB migrations (server/database/migrations/).
- The wedance.vip → 2026 event and profile sync (scripts/sync-v3-events.ts, server/utils/v3EventSync.ts, server/utils/v3ProfileSync.ts), because it writes to production daily.
- Functional, access-control, accessibility (WCAG 2.2 AA on key pages), and basic performance (page-load budgets) testing.
- Desktop Chrome and mobile viewport (Pixel/iPhone size) — RAZ-261 was a mobile-only escape.

Out of scope

- wedance.vip (v3) and wedance-v4 themselves — v3 is treated as an external data source, validated only at the sync boundary.
- Stripe and TicketTailor internals — card data never touches WeDance servers; we test our webhook handling and our redirect/return flows only.
- Load testing beyond a page-load budget — current traffic does not justify it; re-evaluate when a festival on-sale is expected to exceed ~100 concurrent buyers.
- Native mobile apps (none exist).

Objectives

1. main is never red for more than one working day: 0 failing and 0 unexplained skipped tests by 2026-11-01 (baseline: 3 failing, 15 skipped).
2. Every PR runs unit tests and a smoke E2E suite as a **required** GitHub check by 2026-11-01 (baseline: no CI).
3. All 8 user journeys (C1–C8) have at least one passing E2E test by 2027-01-31; the 4 CRITICAL flows by 2026-12-06 (baseline: 0 of 8).
4. Escaped defects (Linear user-report + Bug, excluding test/duplicate) fall from ~7.5/month to under 4/month by 2027-01-31, and sign-in/access-control escapes to 0/month.

3. Test Levels & Types

Level	What It Validates	Owner	Framework	Target Share / Count	Run Frequency
Unit	Router logic against the hermetic FakeDb, utils, validation schemas, webhook parsing	Forge (author of the PR)	Vitest 4	~75% · ~320 tests	Every PR
Integration	Routers and migrations against a real Postgres 17 (the 15 skipIf(!DATABASE_URL) tests today); v3 sync upserts	Forge	Vitest + Postgres service container + dev:db-proxy	~17% · ~70 tests	Every PR
E2E (BDD)	One scenario per critical journey flow through the real app, written as Gherkin	Vitali writes scenarios, Forge writes steps	Playwright + playwright-bdd	~8% · ~30 scenarios	@smoke every PR, full nightly
Production smoke	Key pages and APIs answer 200 after a deploy — read-only	Forge	Playwright (request + page)	~8 checks	Every production deploy
Accessibility	WCAG 2.2 AA on home, city, festival, event, sign-in pages	Vitali	@axe-core/playwright	5 pages	Nightly
Security	Dependency vulnerabilities; access-control matrix (which procedures need sign-in)	Forge	bun audit / GitHub Dependabot; unit tests on protectedProcedure / adminProcedure	Per router	Every PR + weekly
Exploratory (manual)	New features before they stabilize, mobile layout, copy and wording	Vitali + Alex	Charter-based sessions, findings as Linear issues	1 session per week	Weekly + before each festival on-sale

Visual regression, contract testing and load testing are deliberately left out at this maturity level (see section 12 for the trigger to add them). Manual exploratory testing stays manual on purpose: several recent escapes were wording or layout problems (RAZ-189 "Datenschutz is not an English word", RAZ-262 overlapping close button) that a person notices and an assertion does not.

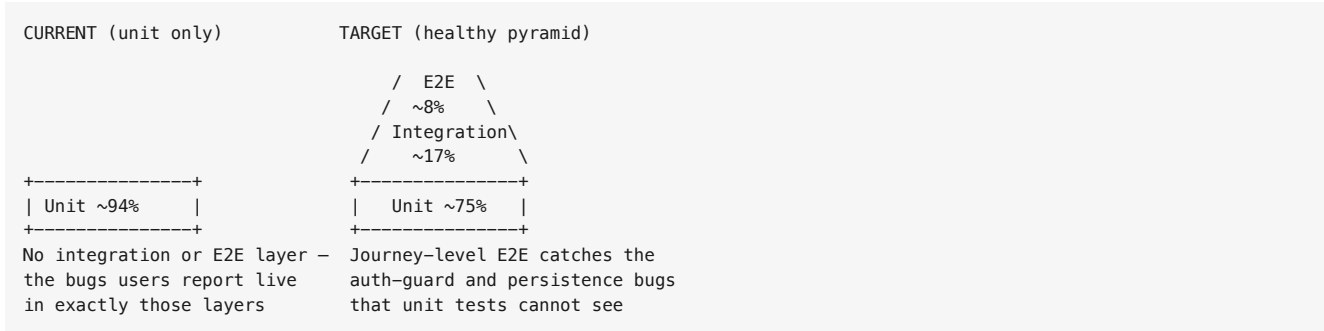
4. Test Pyramid Analysis

Current state (measured 2026-10-09 on origin/main, bunx vitest run):

Current Test Distribution:
Unit tests: 227 run (+15 skipped) → ~94 %
Integration tests: 15 (all skipped – need DATABASE_URL, never run) → ~6 %
E2E tests: 0 runnable (2 orphaned BDD step files) → 0 %
Manual checks: 2 scripts (e2e/synced-events.check.ts, synced-profiles.check.ts)

Current Shape: ☐ Pyramid ☐ Ice Cream Cone ☐ Diamond ☐ Hourglass ☒ No Shape (unit only)

CI Pipeline Duration: none (no CI) – local unit run takes ~10 s
Flaky Test Rate: unknown (never run repeatedly)
Test Suite Pass Rate: 98.7 % (224 of 227 executed tests pass; 3 fail on main)



The failing tests are `auth.me` returns the extended profile fields (the response gained a field the test does not expect) and two `ProfileSettingsSchema` cases in `shared/validation/forms.test.ts` (a new `danceLevels` key). Both are test drift after feature changes — a symptom of having no CI, not of broken features. The orphaned E2E config reads `../product/meetup-planner/scenarios/*.feature`; those files now live in `~/Orgs/WeDance/01_Domains/Festival_Experience/Product/meetup-planner/scenarios/` and describe the older meetup-planner product.

Diagnosis. The suite is not an ice-cream cone; it is a base with nothing on top. That matches the escape data: of the 15 production defects in the last 60 days, none were logic errors a router unit test would have caught. They were UI flows that skipped a sign-in guard (RAZ-230, RAZ-191), state that did not persist across a reload (RAZ-265, RAZ-256), a broken password-reset round trip (RAZ-241), and a production build that broke on main (RAZ-203). Those failure classes live in the E2E and integration layers.

Target state (2027-01-31):

Target Test Distribution:	
Unit:	75% → target count: ~320
Integration:	17% → target count: ~70
E2E:	8% → target count: ~30 scenarios (8 @smoke)
Target CI Duration: < 10 minutes (PR gate)	
Target Flaky Rate: < 2 %	

Action plan (no-shape → pyramid):

1. Make CI exist before adding tests: one GitHub Actions workflow running `bun run test`, made a required status check on main. Fix the 3 failing tests in the same PR.
2. Turn on the integration layer that already exists: run the 15 skipped tests against a Postgres 17 service container in CI, seeded from migrations.
3. Move E2E scenarios into the repo (`e2e/features/`), delete the orphaned path, and write @smoke scenarios for the four CRITICAL flows first.
4. Add a rule to the PR template: a PR that fixes a user-report bug ships a test at the lowest level that would have caught it.

5. Risk Assessment

Score = Impact (1 Negligible → 5 Catastrophic) × Likelihood (1 Rare → 5 Almost Certain). Likelihood uses evidence: escaped defects in the last 60 days, test coverage today, and how often the code changes.

LIKELIHOOD → IMPACT ↓	Rare 1	Unlikely 2	Possible 3	Likely 4	Almost Certain 5
Catastrophic (5)	5-MED	10-HIGH	15-CRIT	20-CRIT	25-CRIT
Major (4)	4-LOW	8-MED	12-HIGH	16-CRIT	20-CRIT
Moderate (3)	3-LOW	6-MED	9-MED	12-HIGH	15-CRIT
Minor (2)	2-LOW	4-LOW	6-MED	8-MED	10-HIGH
Negligible (1)	1-LOW	2-LOW	3-LOW	4-LOW	5-MED

Risk-to-Testing Action Map

Risk Level	Testing Action	Automation	Monitoring
CRITICAL (15–25)	Unit + integration + @smoke E2E + exploratory before releases	Mandatory, every PR	Sentry alert + production smoke
HIGH (10–14)	Unit + integration + E2E in the nightly suite	Mandatory, every PR (unit/integration)	Sentry weekly review
MEDIUM (5–9)	Unit on logic + one happy-path E2E	Recommended	Weekly review
LOW (1–4)	Exploratory sessions only	Optional	None

Feature mapping by user journey

Journey	Flow	Impact	Likelihood	Score	Evidence	Testing Approach
C8 Member	Sign up, sign in, sign out, password reset, session	5	4	20 – CRIT	RAZ-241, RAZ-239, RAZ-240 escaped; 3,900 migrated accounts depend on Firebase-script continuity	Unit (auth router, exists) + integration (sessions table) + @smoke E2E for each step
C8 Member	Access control on write actions (Going, plan, ride, room, hangout)	4	4	16 – CRIT	RAZ-230, RAZ-191 escaped — server procedures are protected, the UI let anonymous users act	Unit: matrix test that every mutating procedure is protectedProcedure/adminProcedure; @smoke E2E: anonymous user is sent to sign-in
C4 Traveler	Ticket purchase via TicketTailor, Stripe referral discount + webhook credit	5	3	15 – CRIT	Real money; webhooks (24 tests) and festivalSignup.ticketCheckout unit-tested; no end-to-end check of the redirect and return flow	Unit (exists) + integration (referral row lifecycle) + @smoke E2E with Stripe test mode up to the checkout redirect
Platform	Production build and deploy of main	5	3	15 – CRIT	RAZ-203: main broke the Vercel build; ~3 merges/day by an automated gate	Required CI check + production smoke after every deploy
C3 Regular	Upcoming events and profiles synced daily from wedance.vip → city and event pages	4	3	12 – HIGH	Writes to production daily (~1,300 events); mapping is unit-tested (33 tests)	Unit (exists) + integration (upsert idempotency, rollback by source) + nightly E2E on a Munich city page; turn the *.check.ts scripts into assertions
C4 Traveler	My plan / my year: going, planning counts, goals	3	4	12 – HIGH	RAZ-265, RAZ-256, RAZ-235, RAZ-255 escaped	Integration (plan router against Postgres) + nightly E2E: add goal → reload → still there
C7 Organizer	Festival submission, claim, TicketTailor verification, insights	4	3	12 – HIGH	festivalSignup (566 lines) and claim tested; insights untested	Unit (exists) + E2E for submit-draft and claim
C5 Pro	Booking requests to artists/venues, availability, gigs	3	3	9 – MED	booking.request (public endpoint) and setAvailability tested; gigs untested	Unit for gigs + rate limit on public booking.request + one happy-path E2E

Journey	Flow	Impact	Likelihood	Score	Evidence	Testing Approach
C8 Member	Community: elections, guideline votes, reviews	4	2	8 – MED	Vote integrity matters; election has 3 tests for 420 lines	Unit on vote counting + one E2E vote
C6 Host	Private event and venue requests (/for-events, entity pages)	3	2	6 – MED	entity router tested via entityBooking.test.ts	Unit (exists) + one E2E request
C1 Seeker	Find your dance quiz, first class, taster	2	3	6 – MED	UI-only logic, frequent copy changes	One E2E happy path + exploratory
C8 Member	City video vote / battles	2	3	6 – MED	cityVideo (452 lines) untested; elo tested	Unit on vote recording
Admin	Group assignment, giveaways, video moderation	3	2	6 – MED	6 admin tests skipped	Integration (unskip)
C2 Student	Teacher and artist discovery	2	2	4 – LOW	Read-only listing	Exploratory
C8 Member	Profile settings and preferences	2	2	4 – LOW	2 failing validation tests (test drift)	Fix tests; exploratory
C8 Member	Giveaways (public entry)	2	2	4 – LOW	Low traffic	Exploratory
Platform	Legal pages (imprint, privacy, terms)	3	1	3 – LOW	Static	Production smoke checks they return 200

Note on the v3 event sync: it scores HIGH, not CRITICAL. A wrong event time is visible to a whole city, but no money or account is lost, its mapping is already the best-tested code in the repo, and rollback is one DELETE ... WHERE source='wedance-v3'. It becomes CRITICAL if 2026 ever becomes the only place those events are published.

6. Environment Strategy

Environment	Purpose	Test Types	Data	Deploy Trigger
Local	Developer (agent) feedback	Unit, integration, E2E	Docker Postgres 17 restored from a production dump without dancers/sessions rows (see CLAUDE.md)	On save / on demand
CI (GitHub Actions)	PR gate	Unit, integration, @smoke E2E against nuxt build + nuxt preview	Ephemeral Postgres service container, migrations + db:seed	Every push to a PR, every push to main
Vercel Preview	Human review of a PR	Exploratory, read-only checks only	Shares the production database	Every PR
Production	Real users	Read-only production smoke, Sentry, PostHog funnels	Live	Every merge to main

The important constraint: **preview deployments share the production DATABASE_URL**. No automated test that writes data may run against a preview or production — a sign-up E2E test there creates real accounts. All write-path E2E tests run in CI against the ephemeral Postgres. This keeps the cost at zero (no Neon branching needed today); if previews ever need write tests, add Neon branch-per-preview first (section 12).

Test data rules: seed data lives in `server/database/seed.ts`; E2E test accounts use the `@test.wedance.vip` email domain so they can never collide with real users; the production dump used locally excludes dancers and sessions table data (personal data, GDPR).

7. Tool Selection

The need: test Nuxt pages and tRPC calls end to end, written by AI agents who already know the stack, with scenarios a non-developer (Vitali, Alex) can read and write.

Criteria (weight)	Playwright + playwright-bdd	Cypress + Cucumber preprocessor	Manual check scripts (status quo)
Fits tech stack (25%)	5	4	3
Team familiarity (20%)	5 — already installed, 2 step files exist	2	4
Community & docs (15%)	5	4	1
CI integration (15%)	5	4	2
Maintenance cost (10%)	4	3	2
Speed of execution (10%)	5	3	4
License cost (5%)	5	4	5
Weighted total	4.90	3.40	2.85

Decision: **keep the stack already in the repo** — Vitest 4 for unit and integration, Playwright with playwright-bdd for E2E (Gherkin scenarios readable by non-developers, which fits the existing BDD skills and docs/issues stories), `@axe-core/playwright` for accessibility, GitHub Actions for CI (the repo is public, so standard runners cost nothing), Sentry for production errors (already configured). Coverage via `@vitest/coverage-v8`, report-only in Phase 1.

CI scaling levers are not needed yet: the unit suite runs in about 10 seconds. Revisit sharding (`--shard`) only if the PR gate passes 10 minutes; at that point add a parallel-efficiency metric to section 10.

Optional framing references for readers who expect a standard: ISTQB CTFL vocabulary and the Heuristic Test Strategy Model; this document is intentionally lighter than a full ISO/IEC/IEEE 29119-3 test plan.

8. Entry/Exit Criteria

Unit — Entry: the PR compiles (`nuxt build` succeeds). Exit: all unit tests pass; no new `skip/skipIf` without a linked Linear issue; new or changed tRPC procedures have at least one test for the allowed caller and one for a rejected caller.

Integration — Entry: unit tests pass; the Postgres service container is up and migrations apply cleanly from zero. Exit: all integration tests pass; every new migration is applied in CI before merge; the 15 currently skipped tests run (not skip) in CI.

E2E — Entry: integration passes; the app is built and served by CI with seeded data and `@test.wedance.vip` accounts. Exit: all `@smoke` scenarios pass on every PR; the full suite passes nightly; no open user-report bug in a CRITICAL flow.

Release (production deploy) — Entry: the PR gate is green and the merge gate has merged to `main`. Exit: the production smoke passes within 10 minutes of the Vercel deploy; Sentry shows no new error type during a 30-minute bake window; if either fails, `main` is reverted (Vercel "promote previous deployment" is the rollback, then a revert PR).

9. Quality Gates

Merging is automated: Forge's Linear gate merges a PR when it has no conflicts and its checks are green. Today the only check is the Vercel build. These gates work by becoming **required GitHub checks**, so the existing automation enforces them with no extra process — a gate that can be clicked past is documentation, not a gate.

PR gate (every PR, target under 10 minutes, required check test):

- `bun run test`: 0 failures.
- Integration job against Postgres: 0 failures.
- `@smoke E2E` against the CI build: 0 failures.
- Vercel build succeeds (existing check).
- Coverage report posted; from Phase 3, line coverage on `server/trpc/routers/` must not decrease.
- No new skipped test without a Linear issue id in the skip reason.

Merge gate (automated Linear gate):

- All required PR-gate checks green on the latest commit.
- No merge conflicts; branch up to date with main (required by branch protection).
- A PR labeled as a `user-report fix` contains a test file change.

Deploy gate (after Vercel production deploy):

- Production smoke, read-only: `/`, `/cities/munich`, `/festivals`, one `/events/[id]`, one `/@handle`, `/api/festivals`, `/api/cities`, the sign-in page → all return 200 and render their main heading.
- Sentry: no new issue with more than 5 events in 30 minutes.
- Fail → revert and open a Bug issue automatically.

Nightly gate (scheduled GitHub Action, 03:00 Europe/Berlin):

- Full E2E suite including mobile viewport.
- Accessibility scan of 5 key pages: 0 new serious/critical axe violations.
- `bun audit / Dependabot`: 0 new high or critical vulnerabilities.
- After the 06:15 v3 sync: `synced-events` check on a Munich sample (no digit-only style tags, no epoch numbers, no timezone shift).
- Results posted to the Linear QA parent (RAZ-268) only when something fails; Vitali reviews failures the next working day.

Definition of Done (shown in the PR template): tests at the right level are in the PR; a bug fix ships the test that would have caught it; no new skipped tests; scenario updated in `e2e/features/` if a journey flow changed.

10. Metrics & KPIs

Metric	Definition	Baseline (2026-10-09)	Target	Cadence
Failing tests on main	Unit + integration failures on the latest main commit	3	0 (never red > 1 working day)	Every push
Skipped tests	Tests marked <code>skip</code> / <code>skipIf</code> that do not run in CI	15	0 by 2026-11-01	Weekly
Journey E2E coverage	User journeys (C1–C8) with ≥1 passing E2E scenario	0 of 8	4 of 8 (CRITICAL flows) by 2026-12-06; 8 of 8 by 2027-01-31	Monthly
Escaped defects	Real user-report + Bug issues per month (no test/duplicate)	~7.5 / month (15 in 60 days)	< 4 / month by 2027-01-31	Monthly
Auth & access-control escapes	Escaped defects in sign-in or sign-in guards	5 in 60 days	0 / month	Monthly
PR gate duration	Push to green/red on the test check	no CI	< 10 min	Weekly

Metric	Definition	Baseline (2026-10-09)	Target	Cadence
Flakiness rate	% of CI runs that fail then pass on rerun with no code change	unknown	< 2 %	Weekly
Router coverage	Line coverage on server/trpc/routers/	14 of 24 routers tested; % unmeasured	All 24 routers have tests; ≥ 80 % lines on auth, festivalSignup, booking, webhooks	Monthly
Time to fix escaped defects	Linear created → merged for user-report bugs in CRITICAL/HIGH flows	to be measured in Phase 1	< 2 working days	Monthly
Production smoke pass rate	Deploys whose smoke passes first time	none	≥ 95 %	Monthly

Targets are set from the measured baseline, not from industry averages. Track trends, investigate spikes (a sudden flakiness jump is usually infrastructure, not carelessness), and never use these numbers to judge individual contributors or agents. Data sources: GitHub Actions (CI metrics), Vitest coverage reports, Linear labels (escapes, time to fix), Sentry (production errors).

11. Timeline & Milestones

Each phase is a Linear milestone in the WeDance project; work items are sub-issues of RAZ-268 labeled qa plus the `cuj:*` journey they cover.

Phase 1 — Foundation (2026-10-12 → 2026-11-01)

- GitHub Actions workflow `test`: unit tests on every PR and on `main`; branch protection makes it required.
- Fix the 3 failing tests; resolve or delete the 15 skipped ones by running them against Postgres in CI.
- Move E2E into the repo (`e2e/features/`), delete the dead `../.. /product/...` path, and write the first `@smoke` scenarios: sign-in, password reset, anonymous user redirected to sign-in on "Going".
- Access-control matrix unit test over all mutating tRPC procedures.
- Remove the stale nested checkouts `wedance-checkout/` and `wedance-2026-raz-158/` from the repo.
- *Exit: the test check is required and green on main; baseline metrics recorded in RAZ-268.*

Phase 2 — Critical coverage (2026-11-02 → 2026-12-06)

- `@smoke` E2E for the remaining CRITICAL flows: ticket purchase up to the Stripe/TicketTailor redirect (test mode), referral credit round trip, production smoke after deploy.
- Integration tests for the plan router and the v3 sync upsert/rollback.
- Nightly workflow with the full E2E suite, mobile viewport, and the synced-events check.
- *Exit: 4 of 8 journeys covered; production smoke runs on every deploy.*

Phase 3 — Gates (2026-12-07 → 2026-12-27)

- Coverage gate (no decrease on routers), PR template Definition of Done, axe accessibility scan in nightly, dependency audit.
- Unit tests for the 10 untested routers by risk: events, hangouts, rideShare, roommate (sign-in guards), festivalInsights, gigs, cityVideo, askLocals, feedback, giveaway.
- *Exit: all four gates enforced; all 24 routers have tests.*

Phase 4 — Full journeys (2026-12-28 → 2027-01-31)

- E2E for C1, C2, C5, C6 happy paths; de-flake anything above the 2 % threshold.
- First quarterly review of this document → version 1.1.
- *Exit: 8 of 8 journeys covered; escaped defects < 4/month; revision 1.1 published.*

Ongoing: weekly exploratory session (Vitali); monthly metrics review in RAZ-268; quarterly revision of this strategy.

12. Risks to the Strategy Itself

Risk	Effect	Mitigation
Tests are written by the same agents that write the code	Tests confirm the implementation instead of the requirement	Vitali writes the Gherkin scenarios from docs/issues stories; agents only implement the steps
The automated merge gate merges before tests exist for a change	Gaps grow faster than coverage	Required checks + Definition of Done in the PR template; the gate already blocks on red
Preview deployments share the production database	A careless E2E run creates real accounts or bookings	Write-path tests only in CI; @test.wedance.vip accounts; a guard in the E2E fixtures that refuses to run write scenarios when BASE_URL is not localhost
E2E flakiness erodes trust	People start ignoring red	Track flakiness weekly; quarantine with a linked issue within 1 day; never retry silently more than once
Feature velocity (~3 merges/day) outpaces scenario writing	Journey coverage stalls	Scenarios prioritized strictly by the section 5 risk order; LOW flows stay manual
QA ownership capacity (one owner)	Reviews and exploratory sessions slip	Exploratory session is 1 hour per week, fixed; nightly failures go to RAZ-268, not to a person's inbox
v3 source becomes the only source of events	Sync risk rises to CRITICAL	Re-score in the quarterly review; add a contract test on the v3 Firestore response shape

Re-evaluation triggers: a CRITICAL-flow escape, a festival on-sale with expected traffic above ~100 concurrent buyers (add load testing), previews needing write tests (add Neon branch-per-preview), a new payment provider, or a change of QA owner.

13. Revision History

Version	Date	Author	Change
1.0	2026-10-09	Forge (drafted with the test-strategy skill) for review by Vitali	Initial strategy: baseline measured on origin/main, risk matrix by user journey, four-phase plan to 2027-01-31

Next scheduled review: 2027-01-31 (end of Phase 4), then quarterly.